```
!pip install -q gradio

import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
model = AutoModelForCausalLM.from_pretrained("ibm-granite/granite-3.3-2b-instruct")
messages = [
    {"role": "user", "content": "Who are you?"},
]
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    tokenize=True,
    return_dict=True,
    return_tensors="pt",
).to(model.device)

outputs = model.generate(**inputs, max_new_tokens=40)
print(tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:]))

!pip install -q gradio

import gradio as gr
import json
import re
from datetime import datetime
from typing import Tuple, Dict, List
import random
```

```
class ColorPaletteGenerator:
    """Generate visually cohesive color palettes"""

    @staticmethod
    def generate_palette(style: str = "modern") -> Dict[str, str]:
        """Generate a cohesive color palette based on style"""

        palettes = {
            'modern': {
                'primary': '#667eea',
                'secondary': '#764ba2',
                'accent': '#f093fb',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'vibrant': {
                'primary': '#ff6b6b',
                'secondary': '#ff8e72',
                'accent': '#f5576c',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'ocean': {
                'primary': '#0099cc',
                'secondary': '#006699',
                'accent': '#00ccff',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
```

```
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'forest': {
                'primary': '#27ae60',
                'secondary': '#1e8449',
                'accent': '#52be80',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'sunset': {
                'primary': '#ff8c42',
                'secondary': '#ff6b35',
                'accent': '#ffa751',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'luxury': {
                'primary': '#1a1a1a',
                'secondary': '#333333',
                'accent': '#FFD700',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'creative': {
                'primary': '#9b59b6',
                'secondary': '#8e44ad',
                'accent': '#e74c3c',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            },
            'minimal': {
                'primary': '#2c3e50',
                'secondary': '#34495e',
                'accent': '#3498db',
                'background': '#0f172a',
                'surface': '#1e293b',
                'text_primary': '#f1f5f9',
                'text_secondary': '#cbd5e1',
                'border': '#334155'
            }
        }

        return palettes.get(style.lower(), palettes['modern'])
```

```
class HTMLPageBuilder:
    """Build styled HTML pages from descriptions"""

    @staticmethod
    def extract_sections(description: str) -> List[str]:
        """Extract sections from description"""
        sections = []

        section_keywords = {
            'hero': ['hero', 'banner', 'landing'],
            'features': ['features', 'services', 'benefits'],
            'about': ['about', 'team', 'company'],
            'portfolio': ['portfolio', 'gallery', 'work', 'projects'],
            'testimonials': ['testimonial', 'reviews', 'feedback'],
            'pricing': ['pricing', 'plans', 'cost'],
            'contact': ['contact', 'form', 'reach'],
            'footer': ['footer', 'bottom']
        }
```

```
        desc_lower = description.lower()
```

```
from typing import Dict, List

class HTMLGenerator:

    @staticmethod
    def detect_sections(description: str) -> List[str]:
        """Detect sections from description"""

        section_keywords = {
            "hero": ["welcome", "hero", "intro"],
            "features": ["features", "services", "benefits"],
            "about": ["about", "company", "who we are"],
            "portfolio": ["portfolio", "work", "projects"],
            "testimonials": ["testimonials", "reviews", "clients"],
            "pricing": ["pricing", "plans", "cost"],
            "contact": ["contact", "email", "phone"]
        }

        desc_lower = description.lower()
        sections = []

        for section_type, keywords in section_keywords.items():
            for keyword in keywords:
                if keyword in desc_lower:
                    sections.append(section_type)
                    break

        if not sections:
            sections = ['hero', 'features', 'contact', 'footer']

        return sections

    @staticmethod
    def generate_meta_tags(title: str, description: str) -> str:
        """Generate SEO meta tags"""

        meta_tags = f"""
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{title}</title>
    <meta name="description" content="{description[:160]}">
    <meta name="theme-color" content="#0f172a">
    <meta property="og:title" content="{title}">
    <meta property="og:description" content="{description[:160]}">
    <meta property="og:type" content="website">
    <meta name="twitter:card" content="summary_large_image">
    <meta name="author" content="HTML Quick-Styler">
"""
        return meta_tags

    @staticmethod
    def generate_section_html(section_type: str, palette: Dict) -> str:
        """Generate HTML for specific section"""

        sections = {

            "hero": """
        <section class="hero">
            <div class="hero-content">
                <h1>Welcome to Your Site</h1>
                <p>Create amazing web experiences</p>
                <button class="btn btn-primary">Get Started</button>
            </div>
        </section>
""",

            "features": """
        <section class="features">
            <h2>Features</h2>
            <div class="features-grid">
                <div class="feature-card">
                    <h3>Fast</h3>
                    <p>Lightning-quick performance</p>
```

```
                </div>
                <div class="feature-card">
                    <h3>Secure</h3>
                    <p>Enterprise-grade security</p>
                </div>
                <div class="feature-card">
                    <h3>Scalable</h3>
                    <p>Grow without limits</p>
                </div>
            </div>
        </section>
    """,

            "about": """
        <section class="about">
            <div class="about-content">
                <h2>About Us</h2>
                <p>We're passionate about creating beautiful, functional websites that help your business grow.</p>
            </div>
        </section>
    """,

            "contact": """
        <section class="contact">
            <h2>Get in Touch</h2>
            <form class="contact-form">
                <input type="text" placeholder="Your Name" required>
                <input type="email" placeholder="Your Email" required>
                <textarea placeholder="Your Message" rows="5" required></textarea>
                <button type="submit" class="btn btn-primary">Send Message</button>
            </form>
        </section>
    """,

            "footer": """
        <footer class="footer">
            <div class="footer-content">
                <p>&copy; 2024 Your Company. All rights reserved.</p>
                <div class="footer-links">
                    <a href="#">Privacy</a>
                    <a href="#">Terms</a>
                    <a href="#">Contact</a>
                </div>
            </div>
        </footer>
    """
        }

        return sections.get(section_type, "")

    @staticmethod
    def generate_css(palette: Dict) -> str:
        """Generate CSS"""

        css = f"""
    <style>

    :root {{
        --primary: {palette.get('primary', '#3b82f6')};
        --secondary: {palette.get('secondary', '#6366f1')};
        --accent: {palette.get('accent', '#22c55e')};
        --background: {palette.get('background', '#0f172a')};
        --surface: {palette.get('surface', '#1e293b')};
        --text-primary: {palette.get('text_primary', '#ffffff')};
        --text-secondary: {palette.get('text_secondary', '#cbd5f5')};
        --border: {palette.get('border', '#334155')};
    }}

    body {{
        font-family: Arial, sans-serif;
        background: var(--background);
        color: var(--text-primary);
    }}

    .hero {{
        padding: 80px;
        text-align: center;
```

```
        background: linear-gradient(135deg, var(--primary), var(--secondary));
    }}

    .features-grid {{
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(250px,1fr));
        gap: 20px;
    }}

    .feature-card {{
        background: var(--surface);
        padding: 20px;
        border-radius: 10px;
    }}

    .footer {{
        text-align: center;
        padding: 30px;
        background: var(--surface);
    }}

    </style>
    """

        @staticmethod
        def generate_css(palette):
            css = f"""
    <style>
    body {{
        font-family: Arial, sans-serif;
        margin:0;
        padding:0;
        background:{palette['background']};
    }}

    header {{
        background:{palette['primary']};
        color:white;
        padding:20px;
    }}

    nav {{
        display:flex;
        justify-content:space-between;
        align-items:center;
    }}

    nav a {{
        color:white;
        margin-left:15px;
        text-decoration:none;
    }}

    section {{
        padding:40px;
    }}

    .features {{
        background:{palette['secondary']};
    }}

    .contact {{
        background:{palette['accent']};
    }}

    footer {{
        text-align:center;
        padding:20px;
    }}
    </style>
    """
            return css
```

```
class HTMLGenerationEngine:
    """Main engine for HTML page generation"""
```

```
    @staticmethod
    def generate_page(title: str, description: str, style: str = "modern") -> Tuple[str, str]:
        """Generate complete HTML page"""

        # Generate color palette
        palette = ColorPaletteGenerator.generate_palette(style)

        # Extract sections
        sections = HTMLPageBuilder.extract_sections(description)

        # Generate meta tags
        meta_tags = HTMLPageBuilder.generate_meta_tags(title, description)

        # Generate sections
        sections_html = '\n\n'.join([
            HTMLPageBuilder.generate_section_html(section, palette)
            for section in sections
            if HTMLPageBuilder.generate_section_html(section, palette)
        ])
```

```
    css = HTMLPageBuilder.generate_css(palette)

    # Build complete HTML
    html = f'''<!DOCTYPE html>
    <html lang="en">
    <head>
    {meta_tags}
    {css}
    </head>
    <body>
        <header>
            <nav>
                <h1>{title}</h1>
                <div>
                    <a href="#features">Features</a>
                    <a href="#about">About</a>
                    <a href="#contact">Contact</a>
                </div>
            </nav>
        </header>

        <main>
    {sections_html}
        </main>
    </body>
    </html>'''

    return html, palette


    generation_history = []


    def generate_html_page(title: str, description: str, style: str = "modern") -> Tuple[str, str, str]:
        """Generate HTML page with styling"""

        if not title.strip() or not description.strip():
            return "", "❌ Please provide both title and description!", ""

        try:
            html_output, palette = HTMLGenerationEngine.generate_page(title, description, style)

            color_info = f"""
    **Color Palette:**
    - Primary: {palette['primary']}
    - Secondary: {palette['secondary']}
    - Accent: {palette['accent']}
    - Background: {palette['background']}
    """

            generation_history.append({
                'title': title,
                'description': description,
                'html': html_output,
                'palette': palette,
                'timestamp': datetime.now().isoformat()
            })
```

```
        return html_output, "✅ HTML page generated successfully!", color_info

    except Exception as e:
        return "", f"❌ Error: {str(e)}", ""
```

```
def get_color_suggestions(style: str) -> str:
    """Get color suggestions based on style"""
    palette = ColorPaletteGenerator.generate_palette(style)

    palette_text = f"""
**Palette for "{style}" style:**
- Primary: {palette['primary']}
- Secondary: {palette['secondary']}
- Accent: {palette['accent']}
- Background: {palette['background']}
- Text: {palette['text_primary']}
"""
    return palette_text

print("🚀 Loading HTML Quick-Styler Advanced...")

with gr.Blocks(title="HTML Quick-Styler - Advanced Edition") as demo:

    gr.Markdown("""
    # 🎨 HTML Quick-Styler Advanced
    ## Professional HTML Page Generator with Auto-Styling

    **Generate production-ready HTML pages in seconds**
    """)

    with gr.Tabs():

        # ============= TAB 1: Page Generator =============
        with gr.TabItem("🎯 Generate Page"):
            with gr.Row():
                with gr.Column(scale=1):
                    gr.Markdown("### 📝 Page Details")

                    page_title = gr.Textbox(
                        label="Page Title",
                        value="Modern Portfolio Website",
                        lines=1
                    )

                    page_description = gr.Textbox(
                        label="Page Description & Sections",
                        value="Hero section with welcome message, features list with 3 items, about section, portfolio gallery, 
                        lines=4
                    )

                    color_style = gr.Dropdown(
                        ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
                        value="modern",
                        label="🎨 Color Style"
                    )

                    generate_btn = gr.Button("✨ Generate HTML", size="lg", variant="primary")

                with gr.Column(scale=1):
                    gr.Markdown("### 📊 Output")

                    html_output = gr.Code(
                        language="html",
                        label="Generated HTML/CSS",
                        lines=20
                    )

                    status_msg = gr.Markdown()
                    color_info = gr.Markdown()

            generate_btn.click(
                generate_html_page,
                inputs=[page_title, page_description, color_style],
                outputs=[html_output, status_msg, color_info]
            )
```

```
# ============= TAB 2: Preview =============
with gr.TabItem("👁️ Live Preview"):
    gr.Markdown("### 🔍 Preview Generated Page")

    preview_button = gr.Button("🔄 Load Preview", size="lg", variant="primary")
    preview_output = gr.HTML(value="<p style='text-align:center; color:#fff; padding:40px;'>No preview loaded. Generate a

    def show_preview(html_text):
        if not html_text.strip():
            return "<p style='text-align:center; color:#fff; padding:40px;'>No HTML provided</p>"
        return html_text

    preview_button.click(
        show_preview,
        inputs=[html_output],
        outputs=[preview_output]
    )

# ============= TAB 3: Color Palettes =============
with gr.TabItem("🎨 Color Palettes"):
    gr.Markdown("### 🌈 Explore Color Schemes")

    with gr.Row():
        style_selector = gr.Radio(
            ["modern", "vibrant", "ocean", "forest", "sunset", "luxury", "creative", "minimal"],
            value="modern",
            label="Choose Style"
        )

        palette_btn = gr.Button("🎨 Get Palette", size="lg", variant="primary")

    palette_display = gr.Markdown()

    palette_btn.click(
        get_color_suggestions,
        inputs=[style_selector],
        outputs=[palette_display]
    )

# ============= TAB 4: Templates =============
with gr.TabItem("📋 Templates"):
    gr.Markdown("### 📝 Quick Templates")

    with gr.Row():
        with gr.Column():
            gr.Markdown("""
            #### Portfolio Website
            Hero, Portfolio Gallery, About, Contact, Footer
            """)
            portfolio_btn = gr.Button("Use Template", size="sm")

            def load_portfolio():
                return ("Portfolio Website",
                        "Hero section with welcome banner, portfolio gallery with 6 items, about section, testimonials, co
                        "modern")

            portfolio_btn.click(load_portfolio, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### SaaS Landing Page
            Hero, Features, Pricing, Testimonials, CTA, Footer
            """)
            saas_btn = gr.Button("Use Template", size="sm")

            def load_saas():
                return ("SaaS Product",
                        "Hero banner, features showcase with 3 items, pricing plans comparison, testimonials, call-to-act
                        "vibrant")

            saas_btn.click(load_saas, outputs=[page_title, page_description, color_style])

        with gr.Column():
            gr.Markdown("""
            #### Company Website
            Header, About, Services, Team, Contact, Footer
            """)
```

```
                    company_btn = gr.Button("Use Template", size="sm")

                    def load_company():
                        return ("Company",
                                "Header with navigation, hero section, about company, services list, team members, contact form, f
                                "ocean")

                    company_btn.click(load_company, outputs=[page_title, page_description, color_style])

        # ============= TAB 5: Guide =============
        with gr.TabItem("📚 Guide"):
            gr.Markdown("""
            # 📖 HTML Quick-Styler Guide

            ## How It Works

            ### 1. Enter Details
            - **Title:** Page name
            - **Description:** Sections you want
            - **Color Style:** Pick a theme

            ### 2. Automatic Detection
            The tool recognizes these keywords:
            - `hero` / `banner` → Hero Section
            - `features` / `services` → Features Grid
            - `about` / `team` → About Section
            - `portfolio` / `gallery` → Portfolio
            - `testimonial` / `reviews` → Testimonials
            - `pricing` / `plans` → Pricing Plans
            - `contact` / `form` → Contact Form
            - `footer` → Footer

            ### 3. Generate & Customize
            - Copy generated HTML
            - Edit text and colors
            - Save as .html file
            - Open in browser

            ## Features
            ✅ Responsive design
            ✅ Modern animations
            ✅ SEO meta tags
            ✅ Professional styling
            ✅ Dark theme
            ✅ Color palettes
            ✅ Copy-paste ready

            ## Color Styles
            - **Modern:** Professional blues
            - **Vibrant:** Bold reds
            - **Ocean:** Calm blues
            - **Forest:** Natural greens
            - **Sunset:** Warm oranges
            - **Luxury:** Elegant blacks
            - **Creative:** Purple artistic
            - **Minimal:** Clean grays
            """)

print("\n✅ HTML Quick-Styler Advanced Ready!\n")
demo.launch(share=True, show_error=True, show_api=False)
```